

Heart Failure Prediction System

using Database Design and Machine Learning



A project report submitted as part of the coursework for Database Systems during the 2nd Semester, 2026.

Submitted By

Student Name	Abdul Mateen
Registration no.	2025-BSCPE-119
Section	A

Supervised By

Dr. Bilal Ahmad

Spring 2026

B.Sc. Computer Engineering
Department of Electrical, Electronics & Telecom Engineering
University of Engineering and Technology Lahore, Faisalabad Campus

P R O J E C T R E P O R T

Heart Failure Prediction System

using Database Design and Machine Learning

T A B L E O F C O N T E N T S

01 DatasetSearching	02 DatabaseDesign	03 ML ModelTraining	04 ModelEvaluation	05 FinalConclusion
-------------------------------	-----------------------------	-------------------------------	------------------------------	------------------------------

01

Dataset Searching & Selection

1.1 Authentic Source.....	1
1.2 Real-World Based Dataset.....	1
1.3 Challenges During Dataset Searching	2
1.4 Analyze Dataset.....	2

2.1 Identify Entities.....	3
2.2 Database and Unnormalized Table Creation	3
2.3 Add Additional Features.....	4
2.4 Entities Tables Creation.....	5
2.5 Insert Data Into Tables.....	6
2.6 Normalization.....	7
2.6.1 1NF — First Normal Form.....	7
2.6.2 2NF — Second Normal Form	7
2.6.3 3NF — Third Normal Form	8
2.7 ER Diagram Through Reverse Engineering.....	10
2.8 MySQL Workbench Implementation.....	11
2.9 MySQL Query to Export Data in CSV	16

03

Machine Learning Model Training Journey

3.1	Creating & Activating Virtual Environment	17
3.2	Importing All Required Libraries	17
3.3	Load Dataset.....	18
3.4	Data Visualization.....	19
3.4.1	Age Distribution	19
3.4.2	Heart Patients Distributions	20
3.5	Handling Missing Values	21
3.6	Encoding Categorical Features	21
3.7	Train-Test-Split	22
3.8	Model Training	22
3.9	Model Prediction	23
3.10	Confusion Matrix Display	23
3.11	Saving Trained Model.....	24

04

Model Evaluation on Inference Script

4.1	Importing Libraries & Load Trained Model.....	25
4.2	Taking Input From User	25
4.3	Prediction on Unseen Data.....	26

05

Final Conclusion & Whole Journey Summary

J O U R N E Y C O V E R E D	K E Y O U T C O M E
From raw data discovery to a fully trained and evaluated ML model — integrated with a normalized MySQL database.	A complete pipeline combining database normalization and machine learning inference on real-world heart failure data.

Project Objectives:

- To identify and select an authentic, real-world heart failure dataset from a credible and verifiable source.
- To analyze and understand the dataset structure, features, and potential challenges during the data searching process.
- To design a fully normalized relational database (1NF, 2NF, 3NF) using MySQL Workbench with proper entity identification, table creation, and data insertion.
- To generate an ER Diagram through reverse engineering and export data in CSV format using MySQL queries.
- To build a complete Machine Learning pipeline including data visualization, preprocessing, encoding, train-test split, model training, and prediction.
- To evaluate model performance using a confusion matrix and test the trained model on unseen data through an inference script.
- To save and deploy the trained model for real-time heart failure prediction based on user input.
- To demonstrate a complete end-to-end system integrating database design and machine learning for heart failure prediction.

1. Dataset Searching & Selection

1.1 Authentic Source:

For this project, I selected my dataset from **Kaggle**, which is a well-known platform widely used by data scientists, researchers, and machine learning practitioners. Kaggle provides datasets that are contributed by both individuals and organizations, and many of them are used in academic research and real-world machine learning projects.

I chose Kaggle because it is a trusted and widely recognized source for open datasets. Before selecting the dataset, I ensured that it had proper documentation, a clear structure, and was relevant to my project requirements. The dataset also had user engagement (such as upvotes and discussions), which helped me evaluate its reliability and usefulness.

1.2 Real-World Based Dataset:

The dataset I selected is based on a real-world scenario, meaning it reflects actual data collected from real observations rather than synthetic or simulated values. This makes the dataset more suitable for analysis because it represents genuine patterns and trends.

Using a real-world dataset allows me to apply theoretical knowledge to practical problems, which improves the accuracy and relevance of my analysis. It also helps in understanding how data behaves in real environments, including inconsistencies and variations.

Official Link:

<https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction/data>

1.3 Challenges During Dataset Searching:

During the dataset searching process, I faced several challenges in selecting an appropriate dataset for my project. One of the main difficulties was that many datasets, although available on different platforms, either did not have proper documentation or lacked authenticity. In some cases, the datasets were not backed by any research paper or reliable source, which made it difficult to verify their credibility.

Another challenge was that some datasets appeared to be artificially generated rather than collected from real-world scenarios. Such datasets were not suitable for my project because they do not accurately represent real-world patterns and can affect the quality of analysis.

Additionally, I also found that some datasets that looked relevant at first were either incomplete or missing important attributes required for meaningful analysis. Because of these issues, I had to carefully evaluate multiple datasets before selecting one that was both authentic and suitable for my project requirements.

1.4 Analyze Dataset:

After selecting the dataset, I analyzed its structure and examined all available features to understand their meaning and role in the dataset. The dataset contains 12 attributes, which are used to predict heart-related conditions. A brief description of each feature is given below:

- **Age:** Represents the age of the patient. It is an important factor in determining the risk of heart disease.
- **Sex:** Indicates the gender of the patient (male or female), which may influence heart disease risk patterns.
- **ChestPainType:** Describes the type of chest pain experienced by the patient. It is a key symptom-related feature.
- **RestingBP:** Represents the resting blood pressure of the patient, measured in mm Hg.
- **Cholesterol:** Shows the serum cholesterol level, which is an important health indicator.
- **FastingBS:** Indicates fasting blood sugar level, which helps in identifying diabetes-related risk factors.
- **RestingECG:** Represents results of the resting electrocardiogram, used to detect heart abnormalities.
- **MaxHR:** Shows the maximum heart rate achieved by the patient during exercise.
- **ExerciseAngina:** Indicates whether the patient experiences chest pain during physical activity.
- **Oldpeak:** Represents ST depression induced by exercise relative to rest, used in cardiac stress analysis.
- **ST_Slope:** Describes the slope of the peak exercise ST segment, an important diagnostic feature.
- **HeartDisease (Target Variable):** This is the output variable indicating whether the patient has heart disease (1 = presence, 0 = absence). It is the main target used for prediction.

2. Database Design Journey

2.1 Identify Entities:

In this step, I carefully studied the dataset and identified the main entity for my database design. Since the dataset is related to heart disease prediction, I considered **Patient** as the main entity. All the attributes in the dataset are linked to this entity. Each record in the dataset represents one patient, so the patient and following are central entities of my database.

- Patient
- Clinical Details
- ECG Details
- Diagnosis
- Risk Assessment

2.2 Database and Unnormalized Table Creation:

After identifying the entity, I created a database in MySQL and defined an initial unnormalized table. At this stage, I included all attributes in a single table without applying any normalization rules. This table contained all the raw data exactly as it was in the dataset.

MySQL Code:

```
CREATE DATABASE heart_normalized_db;
USE heart_normalized_db;

CREATE TABLE heart_main (
  id INT PRIMARY KEY,
  Age INT,
  Sex VARCHAR(5),
  ChestPainType VARCHAR(10),
  RestingBP INT,
  Cholesterol INT,
  FastingBS INT,
  RestingECG VARCHAR(20),
  MaxHR INT,
  ExerciseAngina VARCHAR(5),
  Oldpeak FLOAT,
  ST_Slope VARCHAR(10),
  HeartDisease INT,
  AgeGroup VARCHAR(20),
  BP_Category VARCHAR(20),
  Cholesterol_Category VARCHAR(20),
  MaxHR_Category VARCHAR(20),
```

```
RiskScore INT,  
RiskLevel VARCHAR(10)  
);
```

2.3 Add Additional Features:

In this step, I reviewed the dataset and checked if any additional useful features could be added for better analysis. I focused on keeping the dataset meaningful and complete. However, I ensured that any added feature should be relevant to the patient's health condition and useful for future analysis.

MySQL Code:

```
USE heart_normalized_db;  
  
SET SQL_SAFE_UPDATES = 0;  
UPDATE heart_main SET AgeGroup = CASE  
    WHEN Age < 40 THEN 'Young'  
    WHEN Age BETWEEN 40 AND 55 THEN 'Middle-Aged'  
    ELSE 'Senior'  
END;  
  
UPDATE heart_main SET BP_Category = CASE  
    WHEN RestingBP < 120 THEN 'Normal'  
    WHEN RestingBP BETWEEN 120 AND 139 THEN 'Elevated'  
    ELSE 'High'  
END;  
  
UPDATE heart_main SET Cholesterol_Category = CASE  
    WHEN Cholesterol < 200 THEN 'Normal'  
    WHEN Cholesterol BETWEEN 200 AND 239 THEN 'Borderline'  
    ELSE 'High'  
END;  
  
UPDATE heart_main SET MaxHR_Category = CASE  
    WHEN MaxHR < 100 THEN 'Low'  
    WHEN MaxHR BETWEEN 100 AND 150 THEN 'Normal'  
    ELSE 'High'  
END;  
  
UPDATE heart_main SET RiskScore =  
    (CASE WHEN Age > 55 THEN 2 ELSE 0 END) +  
    (CASE WHEN Sex = 'M' THEN 1 ELSE 0 END) +  
    (CASE WHEN FastingBS = 1 THEN 2 ELSE 0 END) +
```

```
(CASE WHEN ExerciseAngina = 'Y' THEN 2 ELSE 0 END) +  
(CASE WHEN Cholesterol > 240 THEN 2 ELSE 0 END) +  
(CASE WHEN RestingBP > 140 THEN 1 ELSE 0 END);  
  
UPDATE heart_main SET RiskLevel = CASE  
  WHEN RiskScore <= 2 THEN 'Low'  
  WHEN RiskScore BETWEEN 3 AND 5 THEN 'Medium'  
  ELSE 'High'  
END;
```

2.4 Entities Tables Creation:

After preparing the unnormalized table, I divided the data into proper structured tables based on the entity. I created a main **Patient** table that included all patient-related attributes. This step helped in organizing the data properly and reducing redundancy in the database.

```
USE heart_normalized_db;  
  
-- Table 1: Patient  
CREATE TABLE patient (  
  PatientID INT PRIMARY KEY,  
  Age INT,  
  Sex VARCHAR(5),  
  AgeGroup VARCHAR(20)  
);  
  
-- Table 2: Clinical Details  
CREATE TABLE clinical_details (  
  ClinicalID INT AUTO_INCREMENT PRIMARY KEY,  
  PatientID INT,  
  RestingBP INT,  
  Cholesterol INT,  
  FastingBS INT,  
  MaxHR INT,  
  Oldpeak FLOAT,  
  BP_Category VARCHAR(20),  
  Cholesterol_Category VARCHAR(20),  
  MaxHR_Category VARCHAR(20),  
  FOREIGN KEY (PatientID) REFERENCES patient(PatientID)  
);  
  
-- Table 3: ECG Details  
CREATE TABLE ecg_details (  
  ECGID INT AUTO_INCREMENT PRIMARY KEY,
```

```
PatientID INT,  
ChestPainType VARCHAR(10),  
RestingECG VARCHAR(20),  
ExerciseAngina VARCHAR(5),  
ST_Slope VARCHAR(10),  
FOREIGN KEY (PatientID) REFERENCES patient(PatientID)  
);  
  
-- Table 4: Diagnosis  
CREATE TABLE diagnosis (  
    DiagnosisID INT AUTO_INCREMENT PRIMARY KEY,  
    PatientID INT,  
    HeartDisease INT,  
    FOREIGN KEY (PatientID) REFERENCES patient(PatientID)  
);  
  
-- Table 5: Risk Assessment  
CREATE TABLE risk_assessment (  
    RiskID INT AUTO_INCREMENT PRIMARY KEY,  
    PatientID INT,  
    RiskScore INT,  
    RiskLevel VARCHAR(10),  
    FOREIGN KEY (PatientID) REFERENCES patient(PatientID)  
);
```

2.5 Insert Data into Tables:

Once the tables were created, I inserted the dataset records into the database tables. I used MySQL queries to import or manually insert the data. This step ensured that all patient records were properly stored in the database for further processing.

```
INSERT INTO patient (PatientID, Age, Sex, AgeGroup)  
SELECT id, Age, Sex, AgeGroup  
FROM heart_main;  
  
INSERT INTO clinical_details (PatientID, RestingBP, Cholesterol, FastingBS, MaxHR, Oldpeak,  
BP_Category, Cholesterol_Category, MaxHR_Category)  
SELECT id, RestingBP, Cholesterol, FastingBS, MaxHR, Oldpeak, BP_Category, Cholesterol_Category,  
MaxHR_Category  
FROM heart_main;  
  
INSERT INTO ecg_details (PatientID, ChestPainType, RestingECG, ExerciseAngina, ST_Slope)  
SELECT id, ChestPainType, RestingECG, ExerciseAngina, ST_Slope
```

```
FROM heart_main;

INSERT INTO diagnosis (PatientID, HeartDisease)
SELECT id, HeartDisease
FROM heart_main;

INSERT INTO risk_assessment (PatientID, RiskScore, RiskLevel)
SELECT id, RiskScore, RiskLevel
FROM heart_main;
```

2.6 Normalization:

To improve the database design, I applied normalization techniques to remove redundancy and improve data integrity.

2.6.1 First Normal Form (1NF):

In this step, I ensured that all attributes contained atomic values and there were no repeating groups. Each column stored a single value, and each record was unique.

MySQL Code:

```
-- 1NF Table

CREATE TABLE normalization_1NF AS

SELECT id AS PatientID, Age, Sex, ChestPainType, RestingBP,
       Cholesterol, FastingBS, RestingECG, MaxHR,
       ExerciseAngina, Oldpeak, ST_Slope, HeartDisease,
       AgeGroup, BP_Category, Cholesterol_Category,
       MaxHR_Category, RiskScore, RiskLevel
FROM heart_main;
```

2.6.2 Second Normal Form (2NF):

After achieving 1NF, I checked for partial dependency. Since the table had a single entity structure, I ensured that all non-key attributes were fully dependent on the primary key.

MySQL Code:

```
-- 2NF Tables
```

```
CREATE TABLE normalization_2NF_patient AS
SELECT id AS PatientID, Age, Sex, AgeGroup FROM heart_main;
CREATE TABLE normalization_2NF_clinical AS
SELECT id AS PatientID, RestingBP, Cholesterol, FastingBS,
       MaxHR, Oldpeak, BP_Category, Cholesterol_Category, MaxHR_Category
FROM heart_main;
CREATE TABLE normalization_2NF_ecg AS
SELECT id AS PatientID, ChestPainType, RestingECG,
       ExerciseAngina, ST_Slope
FROM heart_main;
CREATE TABLE normalization_2NF_diagnosis AS
SELECT id AS PatientID, HeartDisease FROM heart_main;
CREATE TABLE normalization_2NF_risk AS
SELECT id AS PatientID, RiskScore, RiskLevel FROM heart_main;
```

2.6.3 Third Normal Form (3NF):

In this step, I removed transitive dependencies. I made sure that non-key attributes were not dependent on other non-key attributes, which improved data consistency and reduced redundancy.

MySQL Code:

```
-- 3NF Tables
CREATE TABLE normalization_3NF_patient AS
SELECT * FROM normalization_2NF_patient;
CREATE TABLE normalization_3NF_clinical AS
SELECT * FROM normalization_2NF_clinical;
CREATE TABLE normalization_3NF_ecg AS
SELECT * FROM normalization_2NF_ecg;
CREATE TABLE normalization_3NF_diagnosis AS
SELECT * FROM normalization_2NF_diagnosis;
CREATE TABLE normalization_3NF_risk AS
SELECT * FROM normalization_2NF_risk;
```

Output Screenshots:

✓	19	00:15:05	USE heart_normalized_db
✓	20	00:15:05	CREATE TABLE patient (PatientID INT PRIMARY KEY, Age INT, Sex VARCHAR(5), AgeGroup...
✓	21	00:15:05	CREATE TABLE clinical_details (ClinicalID INT AUTO_INCREMENT PRIMARY KEY, PatientID INT, ...
✓	22	00:15:05	CREATE TABLE ecg_details (ECGID INT AUTO_INCREMENT PRIMARY KEY, PatientID INT, Ch...
✓	23	00:15:05	CREATE TABLE diagnosis (DiagnosisID INT AUTO_INCREMENT PRIMARY KEY, PatientID INT, ...
✓	24	00:15:05	CREATE TABLE risk_assessment (RiskID INT AUTO_INCREMENT PRIMARY KEY, PatientID INT, ...

✓	25	00:15:05	INSERT INTO patient (PatientID, Age, Sex, AgeGroup) SELECT id, Age, Sex, AgeGroup FROM heart_main
✓	26	00:15:05	INSERT INTO clinical_details (PatientID, RestingBP, Cholesterol, FastingBS, MaxHR, Oldpeak, BP_Catego...
✓	27	00:15:05	INSERT INTO ecg_details (PatientID, ChestPainType, RestingECG, ExerciseAngina, ST_Slope) SELECT i...
✓	28	00:15:05	INSERT INTO diagnosis (PatientID, HeartDisease) SELECT id, HeartDisease FROM heart_main
✓	29	00:15:05	INSERT INTO risk_assessment (PatientID, RiskScore, RiskLevel) SELECT id, Risk Score, RiskLevel FRO...

✓	62	00:24:14	CREATE TABLE normalization_1NF AS SELECT id AS PatientID, Age, Sex, ChestPainType, RestingBP, ...	918 row(s) affected	Records: 918	Duplicates: 0	Warnings: 0
✓	63	00:24:14	CREATE TABLE normalization_2NF_patient AS SELECT id AS PatientID, Age, Sex, AgeGroup FROM hear...	918 row(s) affected	Records: 918	Duplicates: 0	Warnings: 0
✓	64	00:24:14	CREATE TABLE normalization_2NF_clinical AS SELECT id AS PatientID, RestingBP, Cholesterol, FastingB...	918 row(s) affected	Records: 918	Duplicates: 0	Warnings: 0
✓	65	00:24:14	CREATE TABLE normalization_2NF_ecg AS SELECT id AS PatientID, ChestPainType, RestingECG, ...	918 row(s) affected	Records: 918	Duplicates: 0	Warnings: 0
✓	66	00:24:15	CREATE TABLE normalization_2NF_diagnosis AS SELECT id AS PatientID, HeartDisease FROM heart_main	918 row(s) affected	Records: 918	Duplicates: 0	Warnings: 0
✓	67	00:24:15	CREATE TABLE normalization_2NF_risk AS SELECT id AS PatientID, RiskScore, RiskLevel FROM heart_...	918 row(s) affected	Records: 918	Duplicates: 0	Warnings: 0
✓	68	00:24:15	CREATE TABLE normalization_3NF_patient AS SELECT * FROM normalization_2NF_patient	918 row(s) affected	Records: 918	Duplicates: 0	Warnings: 0
✓	69	00:24:15	CREATE TABLE normalization_3NF_clinical AS SELECT * FROM normalization_2NF_clinical	918 row(s) affected	Records: 918	Duplicates: 0	Warnings: 0
✓	70	00:24:15	CREATE TABLE normalization_3NF_ecg AS SELECT * FROM normalization_2NF_ecg	918 row(s) affected	Records: 918	Duplicates: 0	Warnings: 0
✓	71	00:24:15	CREATE TABLE normalization_3NF_diagnosis AS SELECT * FROM normalization_2NF_diagnosis	918 row(s) affected	Records: 918	Duplicates: 0	Warnings: 0
✓	72	00:24:15	CREATE TABLE normalization_3NF_risk AS SELECT * FROM normalization_2NF_risk	918 row(s) affected	Records: 918	Duplicates: 0	Warnings: 0

TableName	Total_Rows
normalization_1NF	918
normalization_2NF_patient	918
normalization_2NF_clinical	918
normalization_2NF_ecg	918
normalization_2NF_diagnosis	918
normalization_2NF_risk	918
normalization_3NF_patient	918
normalization_3NF_clinical	918
normalization_3NF_ecg	918
normalization_3NF_diagnosis	918
normalization_3NF_risk	918

TableName	Total_Rows
heart_main	918
patient	918
clinical_details	918
ecg_details	918
diagnosis	918
risk_assessment	918

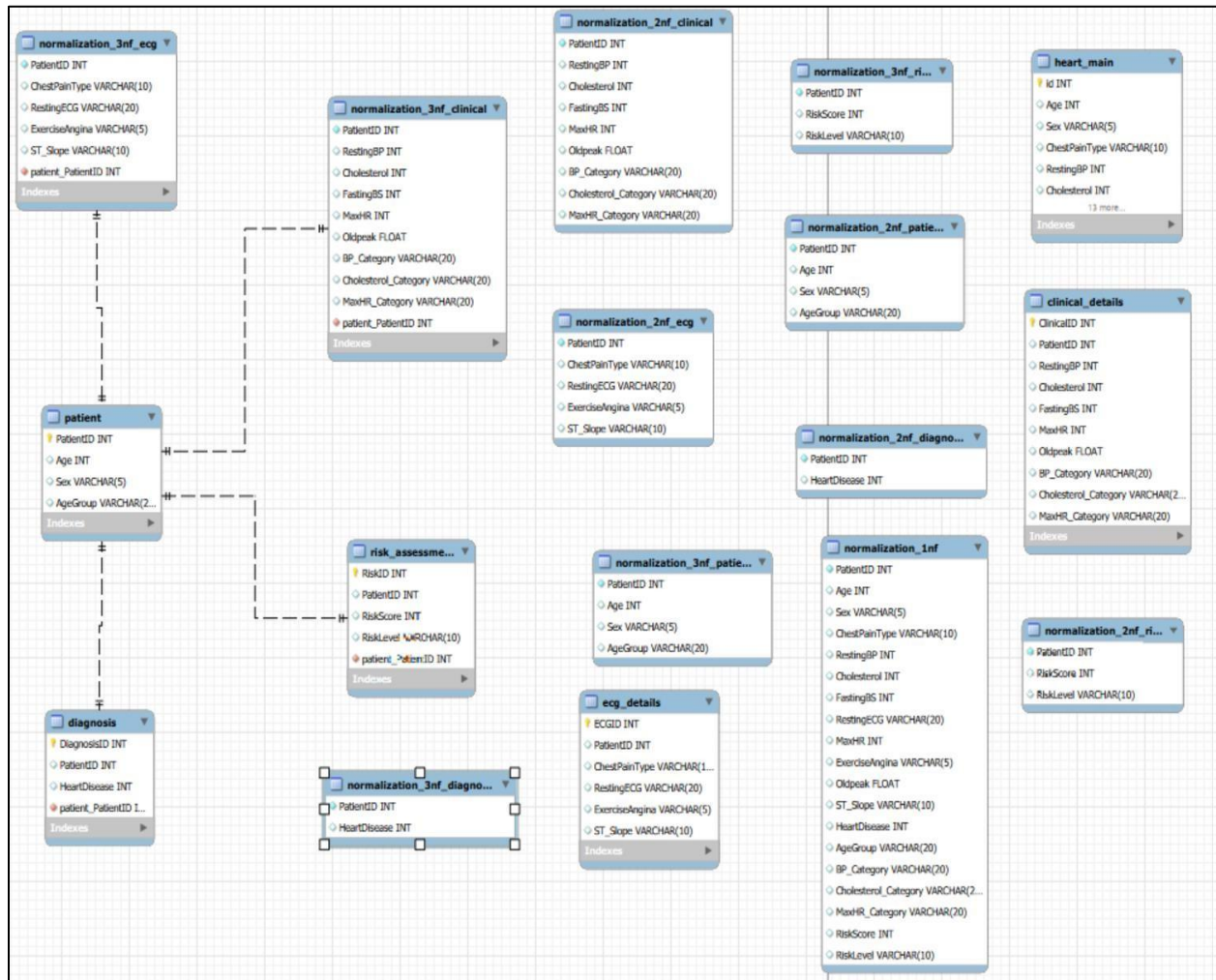
Explanation:

Upto this step most of our database structure and the design is completed because we have successfully created a database followed by patient table creation having all features in it and then started the main purpose of learning this course which is designing relational database management system by creation of multiple tables and finally normalize them to reduce any chances of data redundancy or repetition.

2.7 ER Diagram through Reverse Engineering:

After designing the database, I used MySQL Workbench to generate the ER diagram through reverse engineering. This helped me visually understand the relationship between the table and its attributes. It also confirmed that my database structure was properly organized.

Diagram:



Explanation:

An Entity Relationship (ER) Diagram is a graphical representation of the database structure that illustrates entities, attributes, and relationships among tables. It helps in visualizing how data is organized and connected within the database. In this project, the ER diagram was designed using five entities: **Patient**, **Clinical Details**, **ECG Details**, **Diagnosis**, and **Risk Assessment**. The **Patient** entity serves as the central table, while the remaining entities are linked through the **PatientID** foreign key, ensuring data integrity and efficient relationship management.

2.8 MySQL Workbench Implementation:

I implemented the complete database in MySQL Workbench. I created the database, designed tables, defined data types, and inserted records. I also tested the database by running different queries to ensure everything was working correctly.

Complete MySQL Implementation Code:

```
CREATE DATABASE heart_normalized_db;
USE heart_normalized_db;

CREATE TABLE heart_main (
  id INT PRIMARY KEY,
  Age INT,
  Sex VARCHAR(5),
  ChestPainType VARCHAR(10),
  RestingBP INT,
  Cholesterol INT,
  FastingBS INT,
  RestingECG VARCHAR(20),
  MaxHR INT,
  ExerciseAngina VARCHAR(5),
  Oldpeak FLOAT,
  ST_Slope VARCHAR(10),
  HeartDisease INT,
  AgeGroup VARCHAR(20),
  BP_Category VARCHAR(20),
  Cholesterol_Category VARCHAR(20),
  MaxHR_Category VARCHAR(20),
  RiskScore INT,
  RiskLevel VARCHAR(10)
);

USE heart_normalized_db;

SET SQL_SAFE_UPDATES = 0;
UPDATE heart_main SET AgeGroup = CASE
  WHEN Age < 40 THEN 'Young'
  WHEN Age BETWEEN 40 AND 55 THEN 'Middle-Aged'
  ELSE 'Senior'
END;

UPDATE heart_main SET BP_Category = CASE
```

```
    WHEN RestingBP < 120 THEN 'Normal'
    WHEN RestingBP BETWEEN 120 AND 139 THEN 'Elevated'
    ELSE 'High'
END;

UPDATE heart_main SET Cholesterol_Category = CASE
    WHEN Cholesterol < 200 THEN 'Normal'
    WHEN Cholesterol BETWEEN 200 AND 239 THEN 'Borderline'
    ELSE 'High'
END;

UPDATE heart_main SET MaxHR_Category = CASE
    WHEN MaxHR < 100 THEN 'Low'
    WHEN MaxHR BETWEEN 100 AND 150 THEN 'Normal'
    ELSE 'High'
END;

UPDATE heart_main SET RiskScore =
    (CASE WHEN Age > 55 THEN 2 ELSE 0 END) +
    (CASE WHEN Sex = 'M' THEN 1 ELSE 0 END) +
    (CASE WHEN FastingBS = 1 THEN 2 ELSE 0 END) +
    (CASE WHEN ExerciseAngina = 'Y' THEN 2 ELSE 0 END) +
    (CASE WHEN Cholesterol > 240 THEN 2 ELSE 0 END) +
    (CASE WHEN RestingBP > 140 THEN 1 ELSE 0 END);

UPDATE heart_main SET RiskLevel = CASE
    WHEN RiskScore <= 2 THEN 'Low'
    WHEN RiskScore BETWEEN 3 AND 5 THEN 'Medium'
    ELSE 'High'
END;

USE heart_normalized_db;

-- Table 1: Patient
CREATE TABLE patient (
    PatientID INT PRIMARY KEY,
    Age INT,
    Sex VARCHAR(5),
    AgeGroup VARCHAR(20)
);

-- Table 2: Clinical Details
CREATE TABLE clinical_details (
```

```
ClinicalID INT AUTO_INCREMENT PRIMARY KEY,  
PatientID INT,  
RestingBP INT,  
Cholesterol INT,  
FastingBS INT,  
MaxHR INT,  
Oldpeak FLOAT,  
BP_Category VARCHAR(20),  
Cholesterol_Category VARCHAR(20),  
MaxHR_Category VARCHAR(20),  
FOREIGN KEY (PatientID) REFERENCES patient(PatientID)  
);  
  
-- Table 3: ECG Details  
CREATE TABLE ecg_details (  
    ECGID INT AUTO_INCREMENT PRIMARY KEY,  
    PatientID INT,  
    ChestPainType VARCHAR(10),  
    RestingECG VARCHAR(20),  
    ExerciseAngina VARCHAR(5),  
    ST_Slope VARCHAR(10),  
    FOREIGN KEY (PatientID) REFERENCES patient(PatientID)  
);  
  
-- Table 4: Diagnosis  
CREATE TABLE diagnosis (  
    DiagnosisID INT AUTO_INCREMENT PRIMARY KEY,  
    PatientID INT,  
    HeartDisease INT,  
    FOREIGN KEY (PatientID) REFERENCES patient(PatientID)  
);  
  
-- Table 5: Risk Assessment  
CREATE TABLE risk_assessment (  
    RiskID INT AUTO_INCREMENT PRIMARY KEY,  
    PatientID INT,  
    RiskScore INT,  
    RiskLevel VARCHAR(10),  
    FOREIGN KEY (PatientID) REFERENCES patient(PatientID)  
);  
  
INSERT INTO patient (PatientID, Age, Sex, AgeGroup)  
SELECT id, Age, Sex, AgeGroup
```

```
FROM heart_main;

INSERT INTO clinical_details (PatientID, RestingBP, Cholesterol, FastingBS, MaxHR, Oldpeak,
BP_Category, Cholesterol_Category, MaxHR_Category)
SELECT id, RestingBP, Cholesterol, FastingBS, MaxHR, Oldpeak, BP_Category, Cholesterol_Category,
MaxHR_Category
FROM heart_main;

INSERT INTO ecg_details (PatientID, ChestPainType, RestingECG, ExerciseAngina, ST_Slope)
SELECT id, ChestPainType, RestingECG, ExerciseAngina, ST_Slope
FROM heart_main;

INSERT INTO diagnosis (PatientID, HeartDisease)
SELECT id, HeartDisease FROM heart_main;

INSERT INTO risk_assessment (PatientID, RiskScore, RiskLevel)
SELECT id, RiskScore, RiskLevel

FROM heart_main;

-- 1NF Table
CREATE TABLE normalization_1NF AS
SELECT id AS PatientID, Age, Sex, ChestPainType, RestingBP,
       Cholesterol, FastingBS, RestingECG, MaxHR,
       ExerciseAngina, Oldpeak, ST_Slope, HeartDisease,
       AgeGroup, BP_Category, Cholesterol_Category,
       MaxHR_Category, RiskScore, RiskLevel FROM heart_main;

-- 2NF Tables
CREATE TABLE normalization_2NF_patient AS
SELECT id AS PatientID, Age, Sex, AgeGroup FROM heart_main;
CREATE TABLE normalization_2NF_clinical AS
SELECT id AS PatientID, RestingBP, Cholesterol, FastingBS, MaxHR, Oldpeak, BP_Category,
Cholesterol_Category, MaxHR_Category
FROM heart_main;
CREATE TABLE normalization_2NF_ecg AS
SELECT id AS PatientID, ChestPainType, RestingECG,
       ExerciseAngina, ST_Slope
```

```
FROM heart_main;

CREATE TABLE normalization_2NF_diagnosis AS
SELECT id AS PatientID, HeartDisease FROM heart_main;

CREATE TABLE normalization_2NF_risk AS

SELECT id AS PatientID, RiskScore, RiskLevel FROM heart_main;

-- 3NF Tables

CREATE TABLE normalization_3NF_patient AS
SELECT * FROM normalization_2NF_patient;

CREATE TABLE normalization_3NF_clinical AS
SELECT * FROM normalization_2NF_clinical;

CREATE TABLE normalization_3NF_ecg AS
SELECT * FROM normalization_2NF_ecg;

CREATE TABLE normalization_3NF_diagnosis AS
SELECT * FROM normalization_2NF_diagnosis;

CREATE TABLE normalization_3NF_risk AS

SELECT * FROM normalization_2NF_risk;
```

Implementation Screenshot:

Tables_in_heart_normalized_db
clinical_details
diagnosis
ecg_details
heart_main
normalization_1nf
normalization_2nf_clinical
normalization_2nf_diagnosis
normalization_2nf_ecg
normalization_2nf_patient
normalization_2nf_risk
normalization_3nf_clinical
normalization_3nf_diagnosis
normalization_3nf_ecg
normalization_3nf_patient
normalization_3nf_risk
patient
risk_assessment

Explanation:

Following tables were produced in result of MySQL Workbench code implementation. The screenshot includes the general tables, unnormalized tables and then normalized tables which shows the beauty of a good designed database.

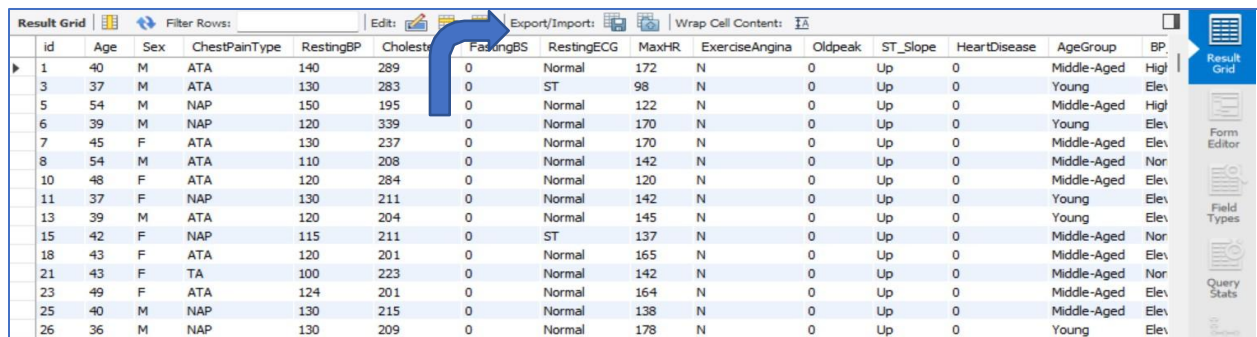
2.9 MySQL Query to Export Data in CSV File:

Finally, I used MySQL queries to export the dataset into a CSV file. This allowed me to use the data outside the database for analysis and reporting purposes. Exporting data also helped in sharing the dataset easily with other tools like Excel or Python.

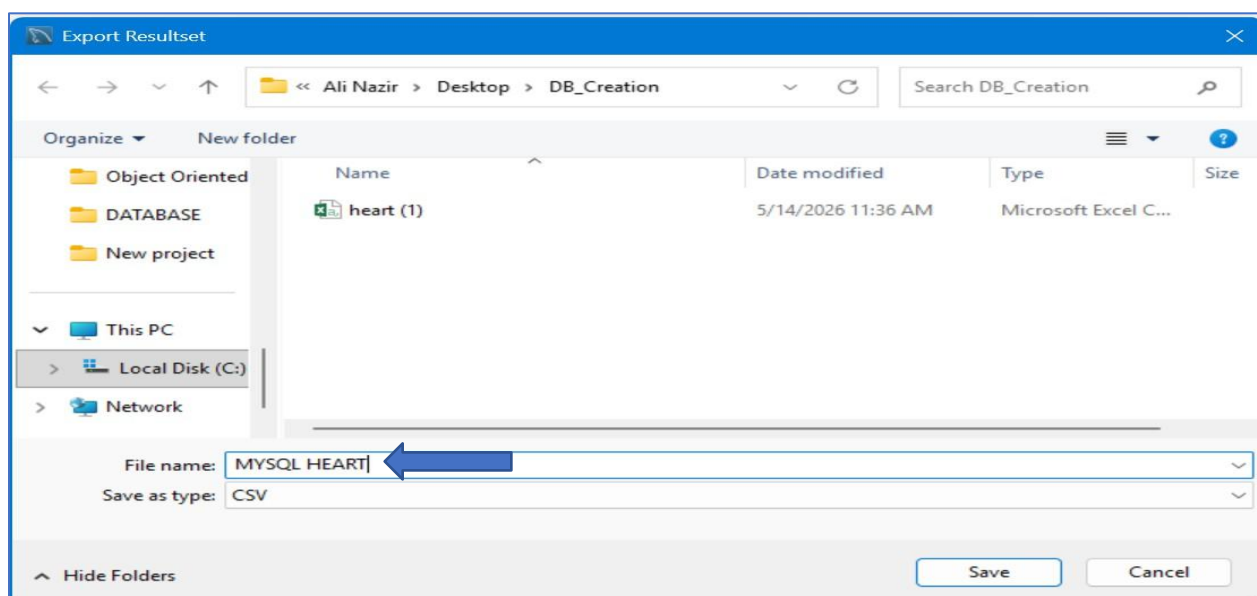
MySQL Query:

```
SELECT * FROM heart_main;
```

Exporting Data into CSV file:



id	Age	Sex	ChestPainType	RestingBP	Cholest	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease	AgeGroup	BP
1	40	M	ATA	140	289	0	Normal	172	N	0	Up	0	Middle-Aged	Higl
3	37	M	ATA	130	283	0	ST	98	N	0	Up	0	Young	Elev
5	54	M	NAP	150	195	0	Normal	122	N	0	Up	0	Middle-Aged	Higl
6	39	M	NAP	120	339	0	Normal	170	N	0	Up	0	Young	Elev
7	45	F	ATA	130	237	0	Normal	170	N	0	Up	0	Middle-Aged	Elev
8	54	M	ATA	110	208	0	Normal	142	N	0	Up	0	Middle-Aged	Nor
10	48	F	ATA	120	284	0	Normal	120	N	0	Up	0	Middle-Aged	Elev
11	37	F	NAP	130	211	0	Normal	142	N	0	Up	0	Young	Elev
13	39	M	ATA	120	204	0	Normal	145	N	0	Up	0	Young	Elev
15	42	F	NAP	115	211	0	ST	137	N	0	Up	0	Middle-Aged	Nor
18	43	F	ATA	120	201	0	Normal	165	N	0	Up	0	Middle-Aged	Elev
21	43	F	TA	100	223	0	Normal	142	N	0	Up	0	Middle-Aged	Nor
23	49	F	ATA	124	201	0	Normal	164	N	0	Up	0	Middle-Aged	Elev
25	40	M	NAP	130	215	0	Normal	138	N	0	Up	0	Middle-Aged	Elev
26	36	M	NAP	130	209	0	Normal	178	N	0	Up	0	Young	Elev



Explanation:

In this task, the data was exported using the MySQL Workbench Result Grid export feature instead of SQL commands. The query results were displayed in the Result Grid, and then exported directly to a CSV file using the “Export Resultset” option. This method provides a simple and user-friendly way to extract data without writing additional SQL export queries. The exported dataset retains all patient records and attributes, making it suitable for preprocessing, feature engineering, model training, and predictive analysis. This step acts as a bridge between database management and machine learning phases of the project.

3. Machine Learning Model Training Journey

3.1 Creating & Activating Virtual Environment:

First, I created a virtual environment in Python to keep my project dependencies separate from other projects. This helped me manage libraries properly without any conflicts. After creating it, I activated the environment so that all installations and work would be done inside it.

Screenshot:

```
PS C:\Users\Ali Nazir\Desktop\DB_PROJECT> python -m venv venv
Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut from Settings > Apps > Advanced app settings > App execution aliases.
PS C:\Users\Ali Nazir\Desktop\DB_PROJECT> python --version
>> py --version
Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut from Settings > Apps > Advanced app settings > App execution aliases.
Python 3.13.13
PS C:\Users\Ali Nazir\Desktop\DB_PROJECT> py -m venv venv
PS C:\Users\Ali Nazir\Desktop\DB_PROJECT> venv\Scripts\activate
(venv) PS C:\Users\Ali Nazir\Desktop\DB_PROJECT>
```

3.2 Importing All Required Libraries:

After setting up the environment, I imported all the necessary Python libraries. These included libraries for data handling, visualization, and machine learning such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn. These libraries helped me in loading data, analyzing it, and building the model.

Code:

```
import pandas as pd

import matplotlib.pyplot as plt
```



```
import seaborn as sns

from sklearn.preprocessing import LabelEncoder

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import joblib
```

Explanation:

- **Pandas:-** Used to load, view and manage the dataset in table format.
- **Matplotlib:-** Used to create basic graphs like histograms, scatter plots and line plots.
- **Seaborn:-** Used to make advanced and attractive graphs like box plots and bar charts.
- **sklearn.preprocessing (LabelEncoder):-** Used to convert text/categorical data into numbers for the model.
- **sklearn.model_selection (train_test_split):-** Used to split the dataset into training and testing parts.
- **sklearn.tree (DecisionTreeClassifier):-** Used to build the machine learning model for predicting Heart disease.
- **sklearn.metrics:-** Used to check model performance with metrics.
- **Joblib:-** Used to save the trained model to a file and load it later for predictions.

3.3 Load Dataset:

Then, I loaded my dataset into the project using Pandas. I read the CSV file and stored it in a DataFrame so I could easily work with it. After loading, I checked the dataset to understand its structure and verify that all data was properly imported.

Code:

```
df = pd.read_csv("heart (1).csv")

print(df.head())

print(df.info())
```

Output:

```
   Age Sex ChestPainType RestingBP Cholesterol ... MaxHR ExerciseAngina Oldpeak ST_Slope HeartDisease
0   40  M           ATA        140         289 ...   172             N         0.0         Up           0
1   49  F           NAP        160         180 ...   156             N         1.0         Flat          1
2   37  M           ATA        130         283 ...    98             N         0.0         Up           0
3   48  F           ASY        138         214 ...   108             Y         1.5         Flat          1
4   54  M           NAP        150         195 ...   122             N         0.0         Up           0

[5 rows x 12 columns]
<class 'pandas.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Age                  918 non-null   int64
1   Sex                  918 non-null   str
2   ChestPainType        918 non-null   str
3   RestingBP            918 non-null   int64
4   Cholesterol          918 non-null   int64
5   FastingBS            918 non-null   int64
6   RestingECG           918 non-null   str
7   MaxHR                918 non-null   int64
8   ExerciseAngina       918 non-null   str
9   Oldpeak              918 non-null   float64
10  ST_Slope             918 non-null   str
11  HeartDisease         918 non-null   int64
dtypes: float64(1), int64(6), str(5)
```

Explanation:

The dataset is loaded from a CSV file using pandas. The `head()` function displays the first few rows, while `info()` provides details about columns, data types, and missing values.

3.4 Data Visualization:

To better understand the dataset, I performed data visualization. This helped me see patterns and distributions in the data.

3.4.1 Age Distribution:

I created a graph to show the distribution of patients' ages. This helped me understand which age groups are more represented in the dataset.

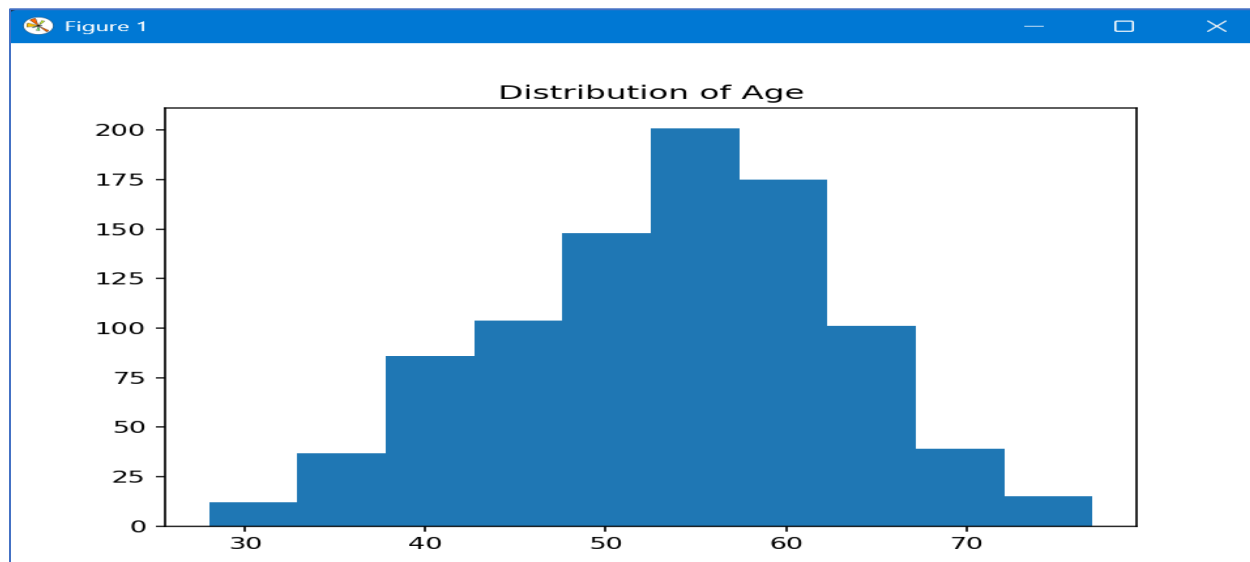
Code:

```
plt.hist(df["Age"])

plt.title("Distribution of Age")

plt.show()
```

Graph:

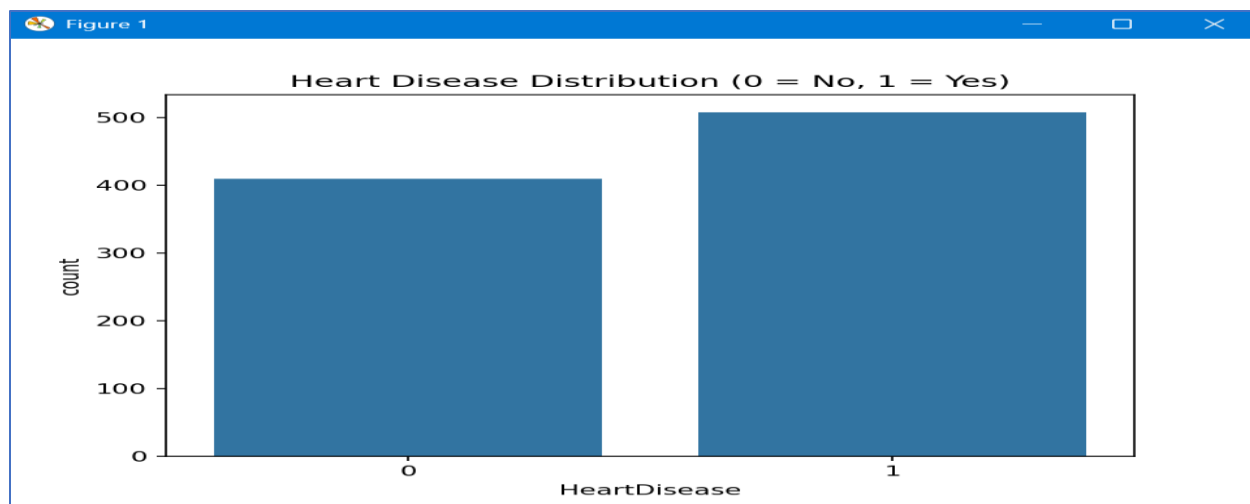


3.4.2 Heart Patients Distribution:

I also visualized the number of patients with and without heart disease. This helped me see whether the dataset was balanced or imbalanced.

Code:

```
sns.countplot(data=df, x="HeartDisease")  
  
plt.title("Heart Disease Distribution (0 = No, 1 = Yes)")  
  
plt.show()
```



Explanation:

This step is used to understand data distribution:

- A histogram is used to visualize the age distribution.
- A countplot shows the number of patients with and without heart disease.

3.5 Handling Missing Values:

After visualization, I checked for missing values in the dataset. If any missing values were found, I handled them by either removing those rows or filling them with appropriate values. This step was important to ensure clean data for training.

Code:

```
for col in df.columns:

    try:

        df[col] = df[col].fillna(df[col].median())

    except TypeError:

        df[col] = df[col].fillna(df[col].mode()[0])
```

Output:

Since there are no missing values in our dataset, so there is no output generated here for this section of our code.

3.6 Encoding Categorical Features:

Since some features were in text format (like Sex, ChestPainType, RestingECG, etc.), I converted them into numerical form using encoding techniques. This step was necessary because machine learning models can only work with numerical data.

Code:

```
for col in df.select_dtypes(include="object"):
    df[col] = LabelEncoder().fit_transform(df[col])

Print(df.Select_dtypes)
```

Output:

eAngina	Oldpeak	ST_Slope	HeartDisease									
0	40	1	1	140	289	...	172	0	0.0	2	0	
1	49	0	2	160	180	...	156	0	1.0	1	1	
2	37	1	1	130	283	...	98	0	0.0	2	0	
3	48	0	0	138	214	...	108	1	1.5	1	1	
4	54	1	2	150	195	...	122	0	0.0	2	0	
...	...	...	...	...	...	...	...	...	...	...	...	...
913	45	1	3	110	264	...	132	0	1.2	1	1	
914	68	1	0	144	193	...	141	0	3.4	1	1	
915	57	1	0	130	131	...	115	1	1.2	1	1	
916	57	0	1	130	236	...	174	0	0.0	1	1	
917	38	1	2	138	175	...	173	0	0.0	2	0	

Explanation:

Machine learning models cannot process text data directly. Therefore, categorical variables are converted into numerical format using Label Encoding. Hence, five of our eleven features those were categorical are encoded in this section.

3.7 Train-Test-Split:

After preprocessing, I split the dataset into training and testing sets. I used most of the data for training the model and kept a portion for testing. This helped me evaluate how well the model performs on unseen data.

Code:

```
X = df.drop("HeartDisease", axis=1)
y = df["HeartDisease"]

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
```

Output:

Since model is split here for training, hence no output is generated for this section of our code.

3.8 Model Training:

Then, I selected a machine learning algorithm and trained the model using the training dataset. During this step, the model learned patterns and relationships between features and the target variable (HeartDisease).

Code:

```
dt = DecisionTreeClassifier()
log_reg = LogisticRegression(max_iter=200)
rf = RandomForestClassifier()
dt.fit(X_train, y_train)
log_reg.fit(X_train, y_train)
rf.fit(X_train, y_train)
```

Output:

Since our model is trained here on provided data, so no output is generated here for this section of our code.

3.9 Model Prediction:

After training, I used the model to make predictions on the test data. The model predicted whether a patient has heart disease or not based on input features.

Code:

```
y_pred = rf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Output:

```
Accuracy: 0.8858695652173914

      precision    recall  f1-score   support

0     0.84      0.90      0.87         77
1     0.92      0.88      0.90        107
```

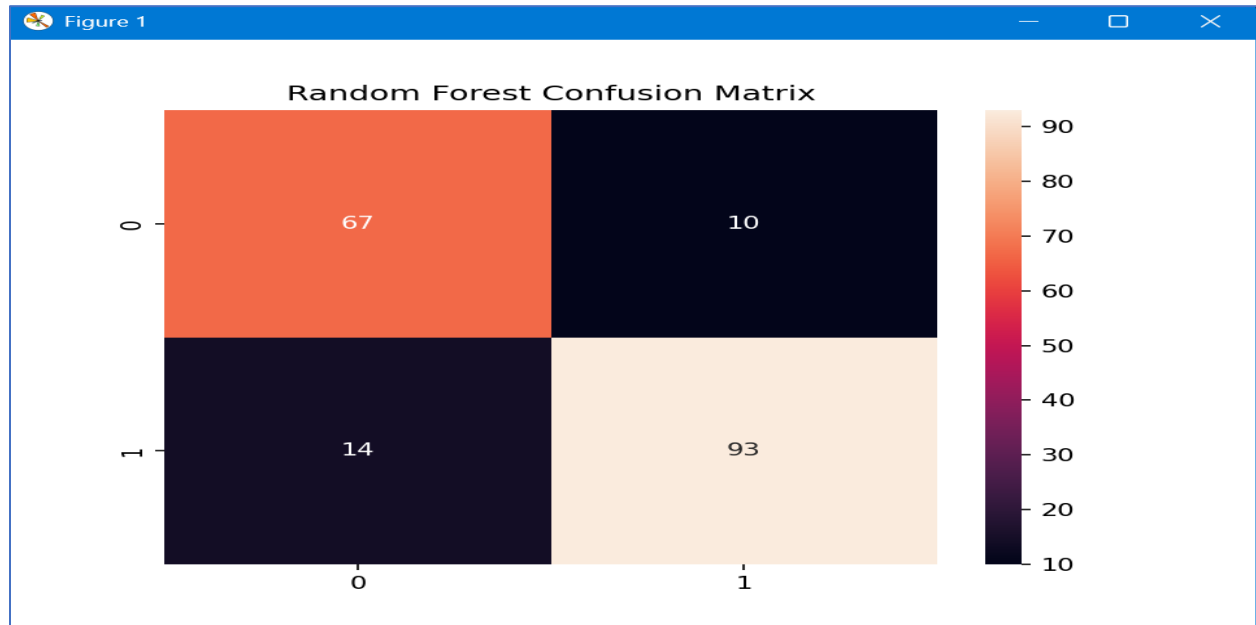
3.10 Confusion Matrix Display:

To evaluate the model's performance, I created a confusion matrix. This helped me compare actual values with predicted values and understand how many predictions were correct or incorrect.

Code:

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d")
plt.title("Random Forest Confusion Matrix")
plt.show()
```

Output:



Explanation:

Confusion matrix is plotted here to visualize the predictions of our model. Confusion matrix visualizes correct and incorrect predictions.

3.11 Saving Trained Model:

Finally, I saved the trained model using a file format like Pickle. This allowed me to reuse the model later without training it again. It also made deployment easier for future use.

Code:

```
joblib.dump(rf, "random_forest_model.pkl")
```

Explanation:

In this task, a complete machine learning workflow was implemented, including data preprocessing, visualization, model training, evaluation, and model saving. The Random Forest model performed the best among all tested models.

4. Model Evaluation on Inference Script

4.1 Importing Libraries & Load Trained Model:

In this step, I started my inference script by importing the required Python libraries. These included libraries for handling data and loading the saved model. After importing the libraries, I loaded my previously trained machine learning model using a file format like Pickle. This allowed me to use the same trained model without training it again.

Code:

```
import joblib
import pandas as pd
model = joblib.load("random_forest_model.pkl")
```

Output:

Since required libraries are imported and model is loaded so there is no output generated here for this section of our code.

4.2 Taking Input From User:

After loading the model, I created a simple input system where I took values from the user. The user entered values for different features such as age, sex, chest pain type, blood pressure, cholesterol, and other health-related attributes. These inputs were necessary to test the model in a real-world scenario.

Code:

```
sample = pd.DataFrame({
    "Age": [int(input("Age: "))],
    "Sex": [int(input("Sex (0=F, 1=M): "))],
    "ChestPainType": [int(input("ChestPainType (0-3): "))],
    "RestingBP": [int(input("RestingBP: "))],
    "Cholesterol": [int(input("Cholesterol: "))],
    "FastingBS": [int(input("FastingBS (0/1): "))],
    "RestingECG": [int(input("RestingECG (0-2): "))],
    "MaxHR": [int(input("MaxHR: "))],
    "ExerciseAngina": [int(input("ExerciseAngina (0/1): "))],
    "Oldpeak": [float(input("Oldpeak: "))],
    "ST_Slope": [int(input("ST_Slope (0-2): "))],
})
```


Output:

```
Age: 69
Sex (0=F, 1=M): 1
ChestPainType (0-3): 3
RestingBP: 150
Cholesterol: 231
FastingBS (0/1): 1
RestingECG (0-2): 0
MaxHR: 162
ExerciseAngina (0/1): 0
Oldpeak: 1
ST_Slope (0-2): 2
```

4.3 Prediction on Unseen Data:

After collecting input from the user, I passed this data to the trained model for prediction. The model analyzed the input features and predicted whether the patient has heart disease or not.

This step is important because it shows how the model performs on completely new and unseen data. It helps in understanding the real-world usability of the model.

Finally, the output was displayed to the user in a simple format, showing either **“Heart Disease: YES”** or **“Heart Disease: NO”** based on the prediction result.

Code:

```
prediction = model.predict(sample)
print("Prediction:", "Heart Disease")
if prediction[0] == 1:
    print("Heart Disease: YES")
else:
    print("Heart Disease: NO")
```

Output:

```
Prediction: Heart Disease
Heart Disease: NO
```

Explanation:

This inference script demonstrates how a trained machine learning model can be used in real-world applications. It takes live user input, processes it, and predicts whether a patient is likely to have heart disease.

5. Final Conclusion and Whole Journey Summary:

In this project, I successfully completed the full cycle of a data science and machine learning pipeline, starting from dataset selection to final model deployment through an inference script. The main objective of this project was to build a machine learning model that can predict the presence of heart disease based on patient health parameters.

At the beginning of the project, I searched for a suitable dataset from a reliable source and selected a real-world dataset related to heart disease. During this phase, I faced challenges such as finding an authentic dataset, ensuring data quality, and avoiding artificially generated or unreliable data. After careful evaluation, I finalized a dataset that was suitable for analysis and modeling.

After dataset selection, I moved towards database design. I identified the main entity as the **Patient** and structured the dataset into a proper database format. I created tables, inserted data, and applied normalization (1NF, 2NF, and 3NF) to reduce redundancy and improve data consistency. I also generated an ER diagram using reverse engineering in MySQL Workbench, which helped me visualize the relationship between data attributes in a clear way.

Next, I began the machine learning model training journey. I set up a virtual environment, imported necessary libraries, and loaded the dataset into Python. I performed exploratory data analysis using visualization techniques such as age distribution and heart disease distribution plots. Then, I handled missing values, encoded categorical features into numerical format, and split the dataset into training and testing sets.

After preprocessing, I trained a machine learning model using the training data. The model learned patterns between input features and the target variable (HeartDisease). I then tested the model on unseen data and evaluated its performance using a confusion matrix. This helped me understand how well the model was performing in terms of correct and incorrect predictions. Finally, I saved the trained model so it could be reused without retraining.

After training, I developed an inference script where I loaded the saved model, took input from users, and performed real-time predictions. This made the project more practical because it demonstrated how the model can be used in real-life scenarios for predicting heart disease risk based on patient data.

Overall, this project gave me hands-on experience in both database design and machine learning implementation.